Intern Software Development Programme

Wireframing & Data Modelling

Project: Help Desk Support System
Revises: Wireframing & Data Modelling
In response to: Help Desk Support System — Project Guide (v2)

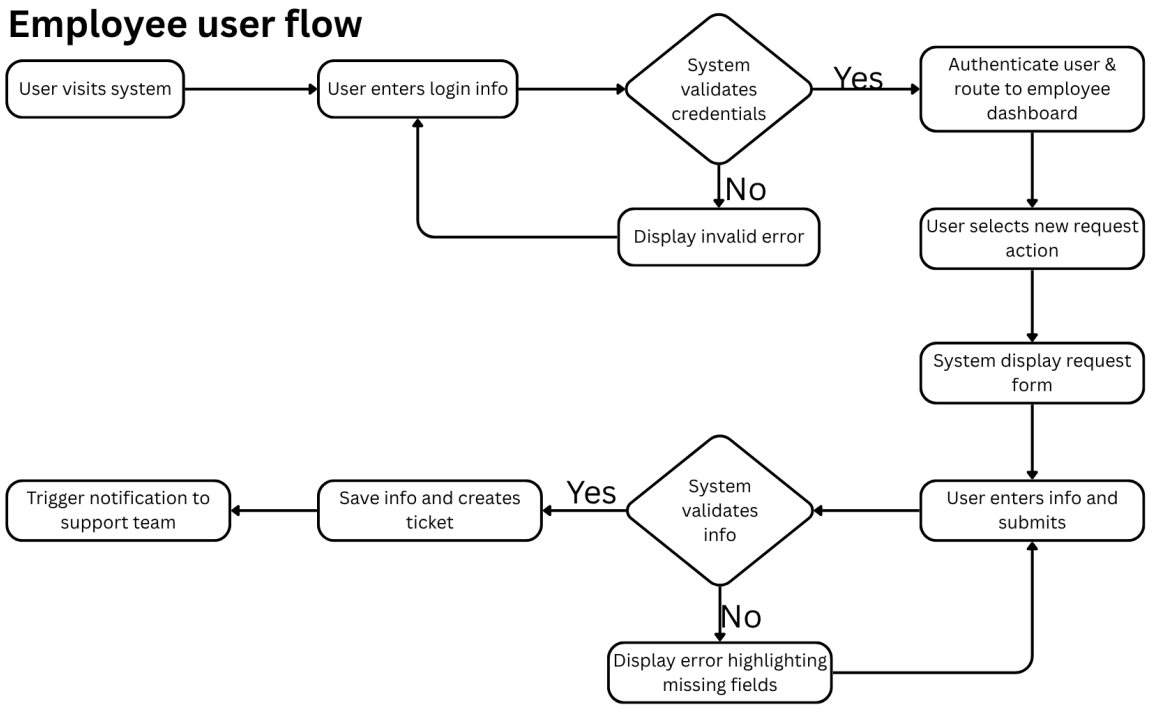
PART 1 — WIREFRAMING

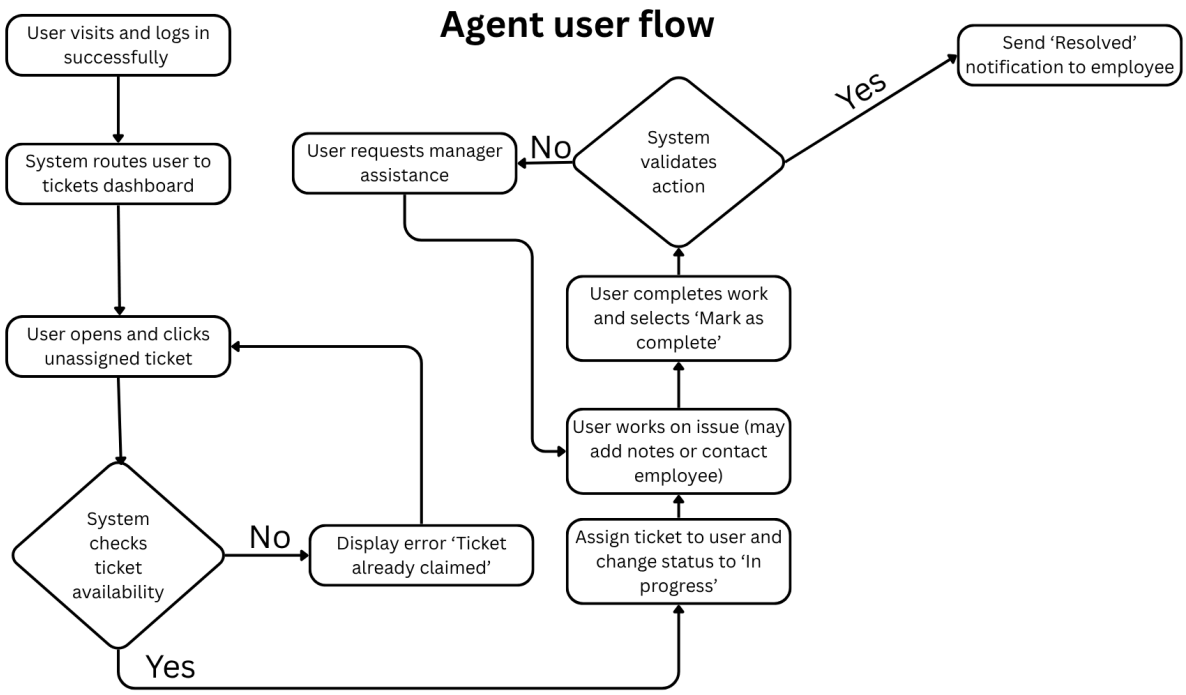
User Roles & Authentication

The system relies on a unified login interface that authenticates users and conditionally routes them based on one of five distinct roles. Passwords must be hashed and never stored in plain text.

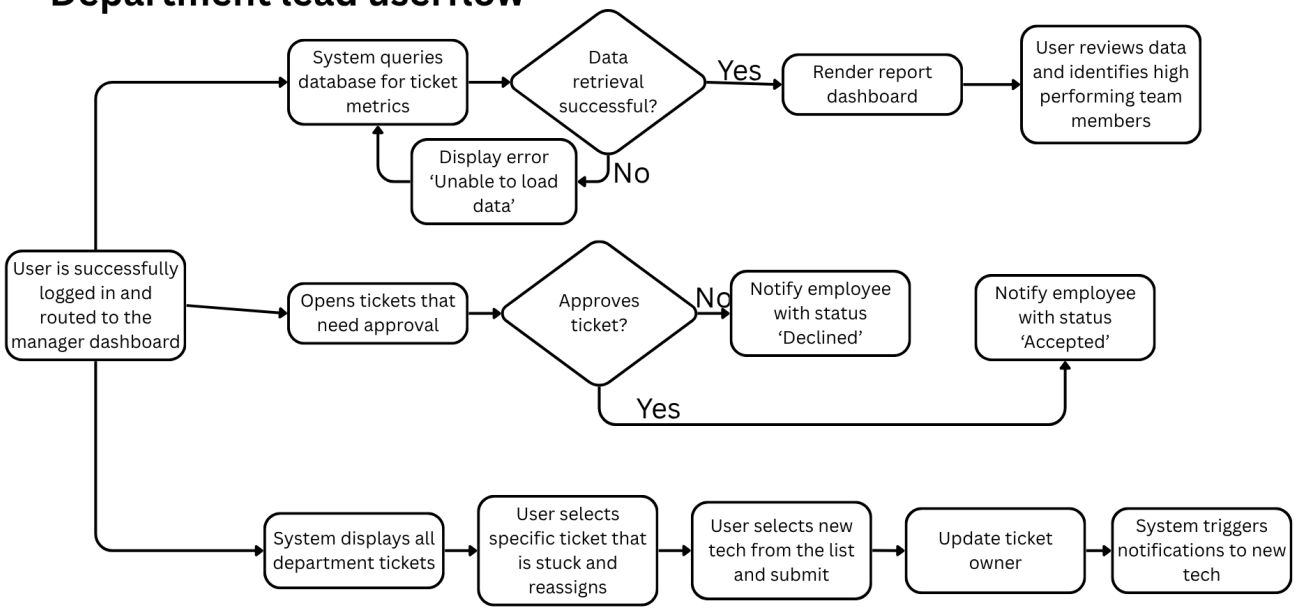
- **Platform Admin:** Responsible for top-level platform access, including creating the initial system admin accounts and resetting locked-out logins.
- **System Admin:** Manages global system settings across all departments. They are responsible for creating departments, appointing Department Leads, managing user accounts (creating or deactivating). System Admin accounts are not scoped to a specific department.
- **Department Lead:** Manages a specific department (e.g., IT, HR, Facilities, Finance). They have access to reporting dashboards, ticket reassignment capabilities, and approval authority for specific request types.
- **Agent:** Operates within a scoped department to resolve tickets.
- **Employee:** The end-user who submits requests and tracks their own ticket statuses.

User Flows

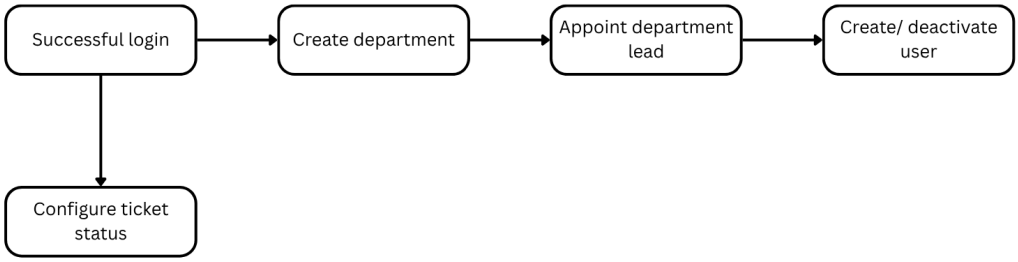

Employee user flow
User visits system
User enters login info
System validates credentials
Yes
Authenticate user & route to employee dashboard
No
Display invalid error
User selects new request action
System display request form
Trigger notification to support team
Save info and creates ticket
Yes
System validates info
User enters info and submits
No
Display error highlighting missing fields


Agent user flow
User visits and logs in successfully
System routes user to tickets dashboard
User opens and clicks unassigned ticket
System checks ticket availability
No
Display error 'Ticket already claimed'
Yes
User requests manager assistance
No
System validates action
Yes
Send 'Resolved' notification to employee
User completes work and selects 'Mark as complete'
User works on issue (may add notes or contact employee)
Assign ticket to user and change status to 'In progress'

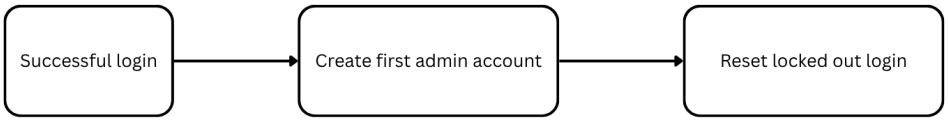
Department lead userflow



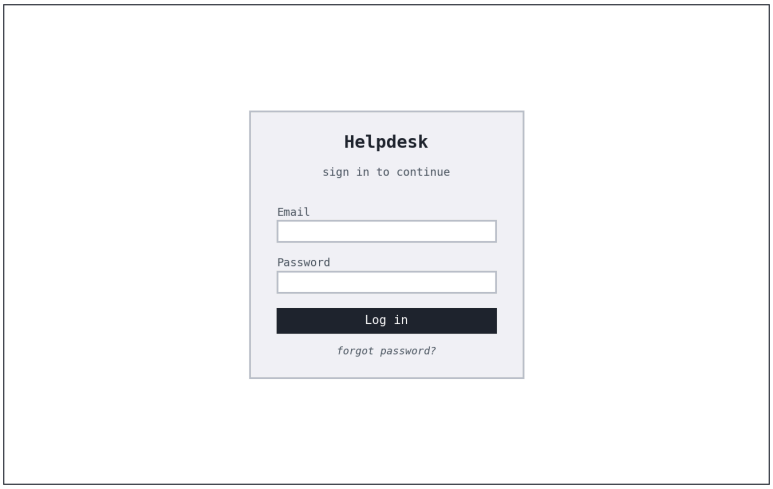
System admin userflow



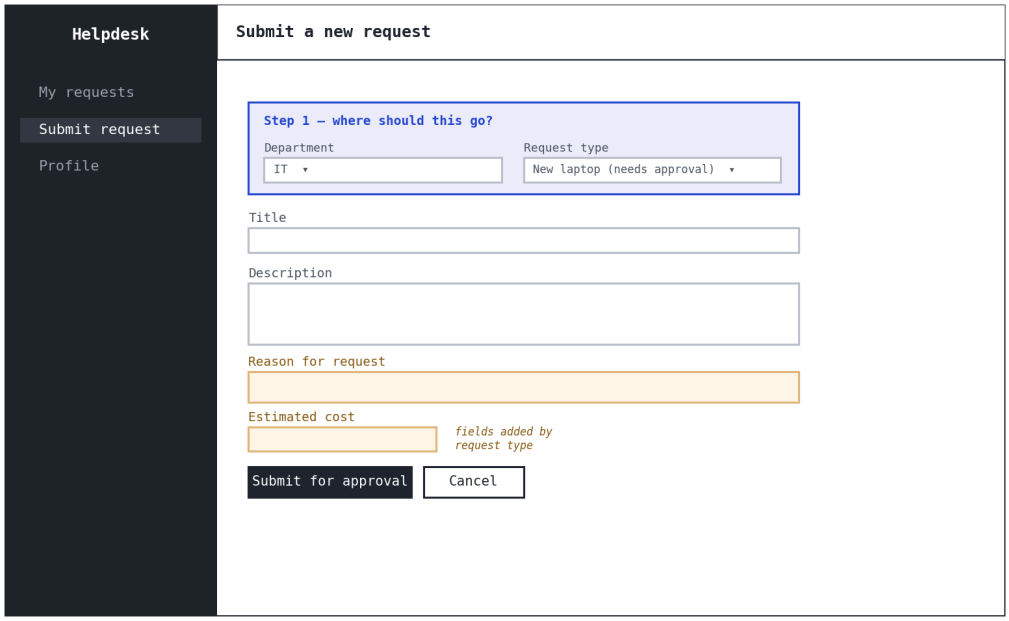
Platform admin userflow



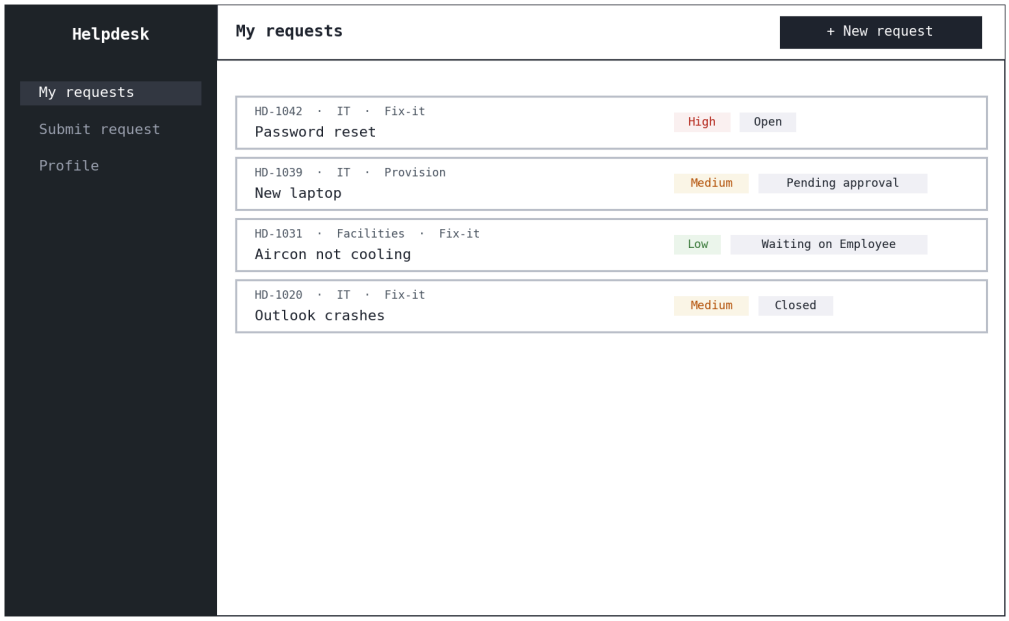
Wireframes



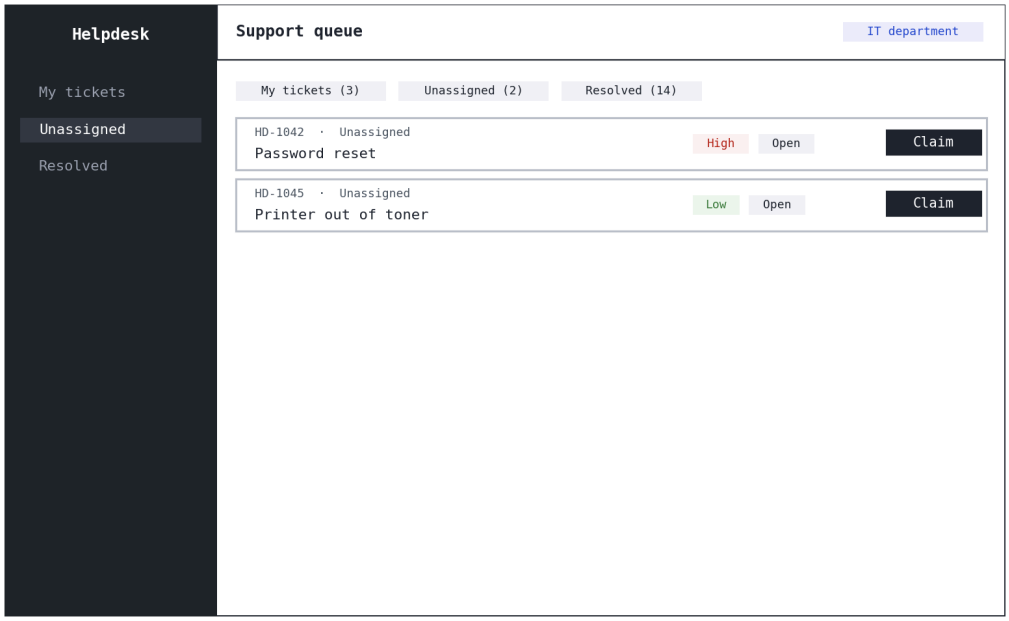
Login — shared entry point for all five roles.



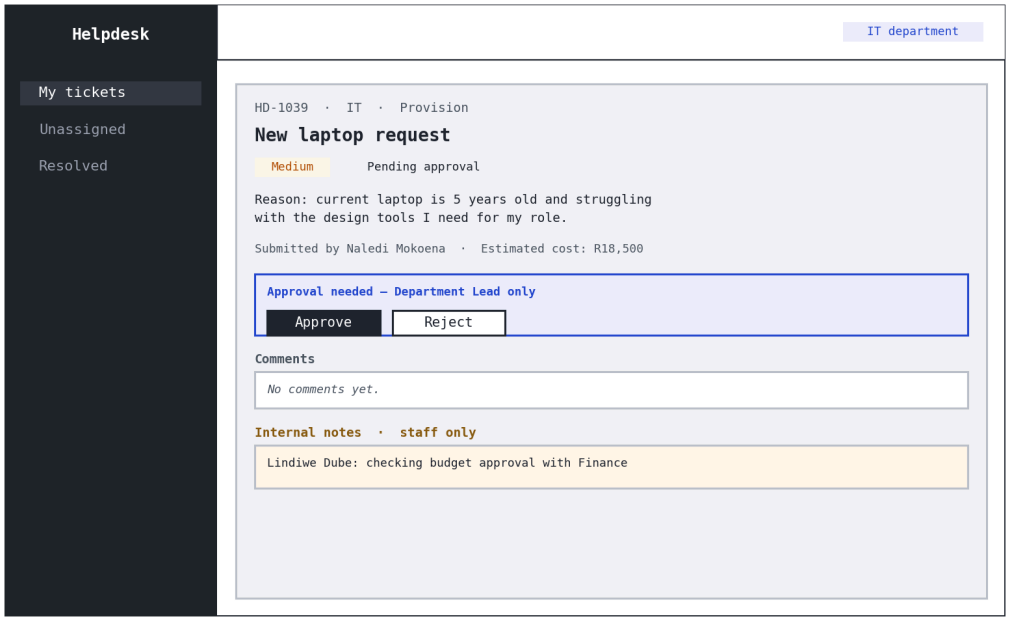
Employee — Submit a new request. NEW: Step 1 department + request-type picker; fields below adapt to the chosen type (shown here: "New laptop", a Provision type, adds Reason and Estimated cost).



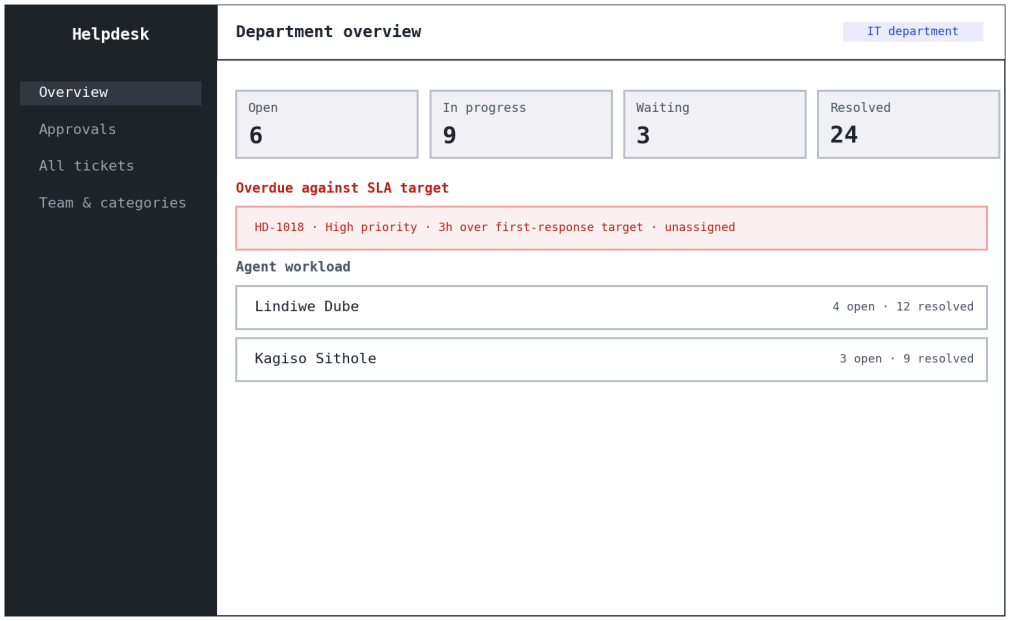
Employee — My requests: each row now shows department, request type, and a priority badge alongside status.



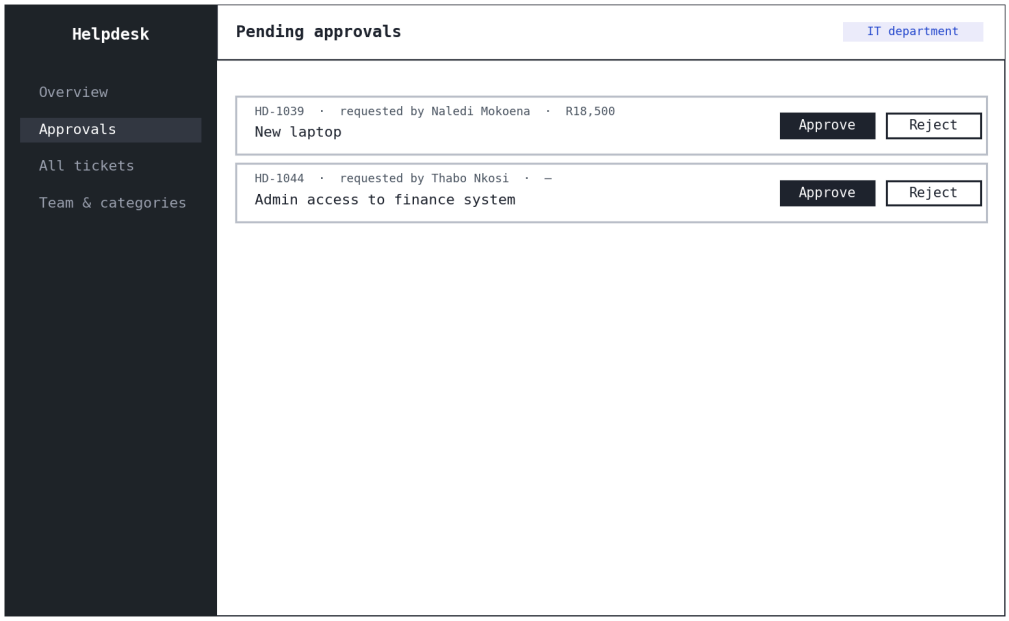
Agent — Support queue: scoped to "IT department" (shown top-right); priority badge added to each row.



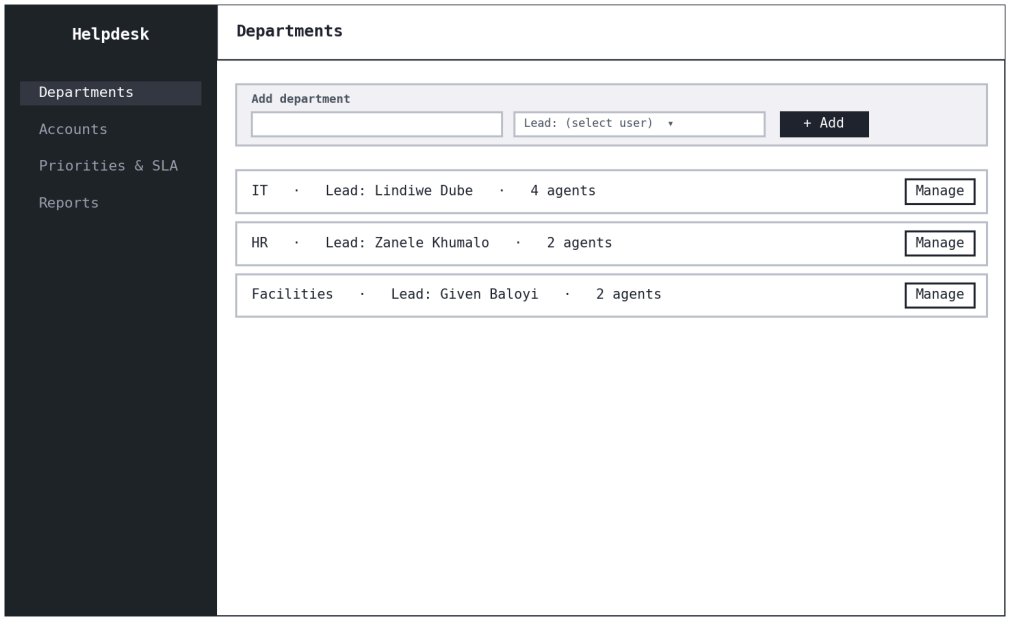
Ticket detail — shown here for a Provision-type ticket. NEW: the "Approval needed — Department Lead only" panel, only rendered for Provision types still Pending Approval.



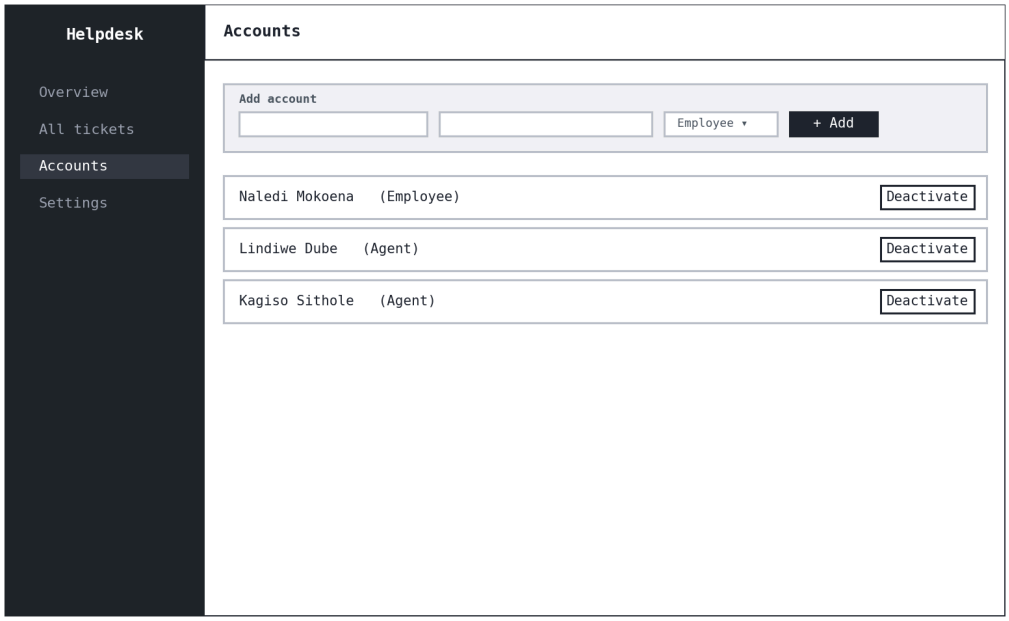
Department Lead — Department overview (was "Team overview"): the overdue-against-SLA panel: scoped to one department.



Department Lead — Pending approvals. NEW screen: every Provision-type request awaiting this department's sign-off, with its Approve/Reject actions.



System Admin — Departments. NEW screen: create a department, appoint its Lead. Accounts, Priorities & SLA, and Reports (not pictured) sit alongside it in the same sidebar.



Accounts — owned by System Admin instead of Manager, and cross-department rather than scoped to one team.

PART 2 — DATA MODELLING

Entity Fields, Keys & Relationships

Department

Field	Type	Key	Notes
department_id	integer	**PK**	Unique identifier.
name	text		e.g. IT, HR, Facilities, Finance.
lead_id	integer	**FK → User, nullable**	The Department Lead; nullable only until a System Admin appoints one.

RequestType

Field	Type	Key	Notes
request_type_id	integer	**PK**	Unique identifier.
department_id	integer	**FK → Department**	Which department this type belongs to.
name	text		e.g. Password reset, New laptop, New-starter setup.
kind	enum		fix_it — goes straight to Open. provision — goes to Pending Approval first.
extra_fields	text		The type's additional form fields (e.g. reason, estimated cost), collected at submission.

Category

Field	Type	Key	Notes
category_id	integer	**PK**	Unique identifier.
department_id	integer	**FK → Department**	Scopes the category to one department.
name	text		e.g. Hardware, Software (IT); Payroll, Leave (HR).

Role

Field	Type	Key	Notes
role_id	integer	**PK**	Unique identifier.
name	enum		employee — agent — department_lead — system_admin — platform_admin.

User

Field	Type	Key	Notes
user_id	integer	**PK**	Unique identifier for the account.
name	text		Display name shown across the system.
email	text		Login identifier; must be unique.
password_hash	text		Never stored in plain text.
role	enum	**FK → Role**	employee — agent — department_lead — system_admin — platform_admin.
department_id	integer	**FK → Department, nullable**	null for system_admin and platform_admin, who aren't scoped to one department.
active	boolean		Set to false instead of deleting an account.
created_at	datetime		Account creation timestamp.

Ticket

Field	Type	Key	Notes
ticket_id	integer	**PK**	Unique identifier, shown to users as e.g. HD-1042.
title	text		Short summary entered on submission.
description	text		Full description entered on submission.
department_id	integer	**FK → Department**	which department's queue this ticket is in.
category_id	integer	**FK → Category**	Which category the request falls under.
request_type_id	integer	**FK → RequestType**	determines extra fields and whether approval is required.
priority	enum		— high — medium — low.
status	enum		pending_approval — open — in_progress — waiting — resolved — closed — cancelled.
requester_id	integer	**FK → User**	The employee who submitted it.
assignee_id	integer	**FK → User, nullable**	The agent who claimed it; empty until claimed.
created_at	datetime		When the ticket was opened.

Field	Type	Key	Notes
resolved_at	datetime, nullable		Set when an agent resolves the ticket.
closed_at	datetime, nullable		set on auto-close (5 days after resolved_at) or employee confirmation.

Comment

Field	Type	Key	Notes
comment_id	integer	**PK**	Unique identifier.
ticket_id	integer	**FK → Ticket**	Which ticket this comment belongs to.
author_id	integer	**FK → User**	Who wrote it — employee or staff.
body	text		The comment text, visible to employee and staff.
created_at	datetime		Timestamp for ordering the thread.

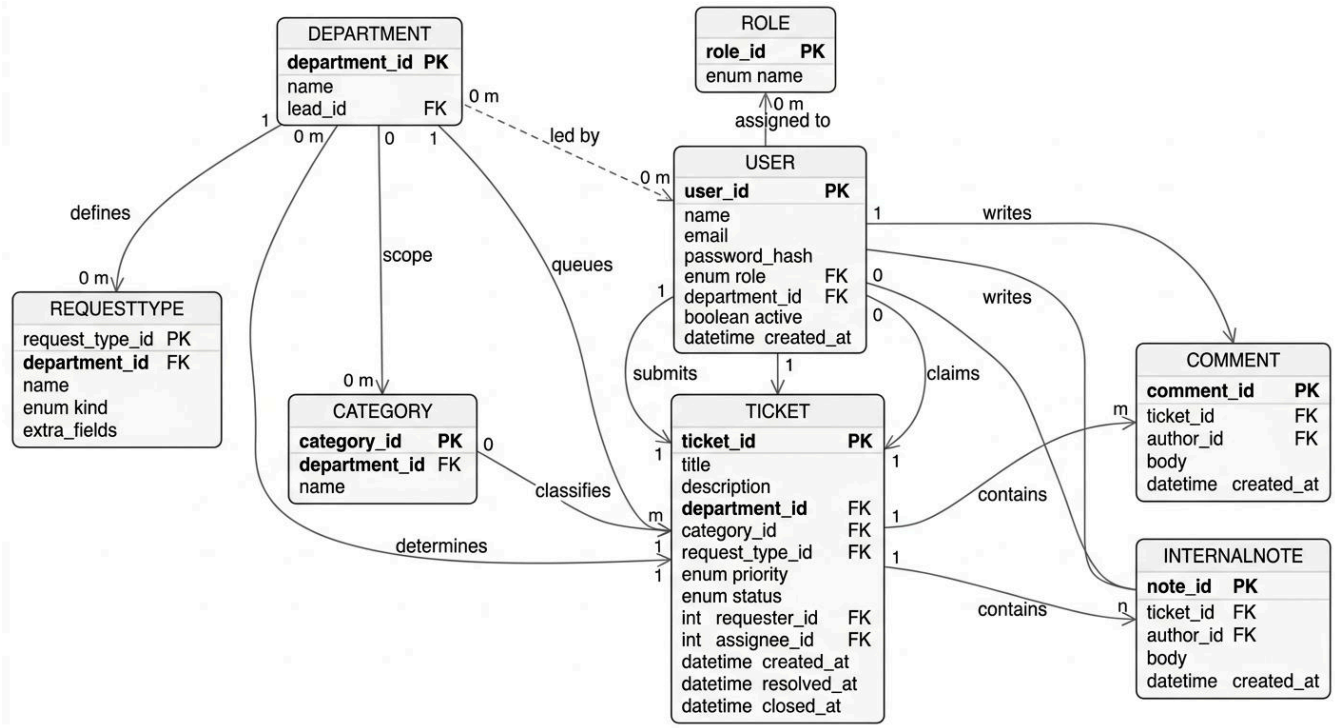
InternalNote

Field	Type	Key	Notes
note_id	integer	**PK**	Unique identifier.
ticket_id	integer	**FK → Ticket**	Which ticket this note belongs to.
author_id	integer	**FK → User**	Agent, Department Lead, or System Admin — never Platform Admin
body	text		Staff-only text, never shown to the requesting employee.
created_at	datetime		Timestamp for ordering the notes.

Relationships

- One Department offers many RequestTypes, and scopes many Categories — one-to-many.
- One Department has many Tickets — one-to-many.
- One RequestType shapes many Tickets — one-to-many.
- One User leads one Department (lead_id) — one-to-one
- One User can submit many Tickets (employee), and be assigned many Tickets (agent)
- One Category applies to many Tickets; one Ticket has many Comments and many InternalNotes; one User authors many Comments and many InternalNotes

Entity Relationship Diagram



PART 3 — CONNECTING WIREFRAMES & DATA

Screen → Data Mapping

Screen	Data shown	User action → backend process
My requests (employee)	Ticket rows where requester_id = current user, joined to Department and RequestType for display	None on load — read-only list, sorted by created_at.
Submit a new request (employee)	Department list; RequestType list filtered by chosen department, with its extra_fields	Submit → validate required + type-specific fields → insert Ticket (status = open if kind = fix_it, else pending_approval).
Support queue (agent)	Ticket rows filtered by department_id = agent's department AND (assignee_id = current user OR assignee_id is null)	Claim → update Ticket.assignee_id and status = in_progress. Move → update Ticket.department_id.
Ticket detail (employee, agent, department lead)	One Ticket, its Comments, its InternalNotes; Approve/Reject panel rendered only if status = pending_approval and viewer is the ticket's Department Lead	Change status → update Ticket.status (and resolved_at / closed_at). Approve/Reject → update status to open or cancelled.
Department overview (department lead)	Ticket counts grouped by status, filtered to department_id = current department; tickets past their priority SLA target	None on load — read-only aggregation.

Screen	Data shown	User action → backend process
Pending approvals (department lead)	Ticket rows where status = pending_approval AND department_id = current department	Approve → status = open. Reject → status = cancelled, reason recorded.
Departments (System Admin)	All Department rows, with lead name and agent count	Add → insert Department. Appoint lead → update Department.lead_id.
Accounts (System Admin)	All User rows across all departments	Add → insert User. Deactivate → update User.active = false. Change role/department → update User.role / department_id.

Team Check

Wireframes

☑ We identified our main screens.
☑ Our screens come from our requirements.
☑ We created wireframes for the important screens, including the three new ones.
☑ We understand how users move between screens.
☑ We know what each screen is supposed to do.

Data model

☑ We identified our main entities/tables, including the two new ones (Department, RequestType).
☑ We identified the important fields, including the new ones on Ticket, Category, and User.
☑ We identified primary keys.
☑ We identified foreign keys where necessary.
☑ We identified relationships.

Connecting the two

☑ We know where the information on our screens comes from.
☑ We know where user-submitted information goes.
☑ We can explain the relationship between screen, user action, backend process and data.
☑ Our wireframes and data model describe the same system.